

The Thirteenth Room - Architecture Notes

Core Gameplay Loop

Player explores a procedurally generated mansion while being hunted by a monster.

Objectives:

- Complete tasks
- Collect useful items
- Maintain mansion systems
- Increase score
- Survive as long as possible

The game ends when the monster successfully attacks the player.

Runtime Startup Flow

GameManager owns the startup sequence.

```
Generate Mansion
→ Spawn Player
→ Spawn Monster
→ Spawn Tasks
→ Spawn Items
→ Begin Gameplay
```

Responsibilities:

```
GameManager
  Controls game state
  Controls startup sequence
  Controls game over

MansionGenerator
  Generates mansion graph
  Spawns room prefabs
  Places doors

MonsterSpawner
  Chooses monster
```

```
Chooses spawn room  
Instantiates monster
```

```
ItemSpawner  
Places items
```

```
TaskSpawner  
Places task-related objects
```

Mansion Architecture

The mansion is represented as a graph.

Room

Room is a node.

Each room stores:

```
RoomId  
  
North  
South  
East  
West  
Up  
Down  
  
PlayerSpawnPoint  
MonsterCenterPoint
```

Rooms may expose:

```
GetNeighbor(direction)  
GetFreeSpawnPoints()
```

Room Types

Necessary Rooms

Must exist.

Examples:

Landing
Monster Spawn Room
Generator Room

Optional Rooms

May appear multiple times.

Examples:

Bedroom
Bathroom
Library
Storage
Office

Room Generation

Generation happens in stages.

Generate logical maze

Assign required rooms

Choose matching room prefabs

Spawn rooms

Place doors

Room Connections

Conceptually:

```
Room = Node  
Connection = Edge
```

Future connection data:

```
RoomConnection  
  
RoomA  
RoomB  
  
Door  
  
DirectionA  
DirectionB
```

May later store:

```
Locked  
Barricaded  
Monster Cost
```

Monster Architecture

MonsterDefinition (ScriptableObject)

Stores monster configuration.

Examples:

```
Name  
Monster Prefab  
  
Allowed Spawn Tags  
  
Base Move Speed
```

Move Interval

Attack Windup

Audio Clips

MonsterDefinition contains NO runtime state.

MonsterController

Stores runtime state.

Examples:

Current Room

Target Room

Current State

States:

Roaming
Travelling
Attacking
Stunned

Monster Spawning

MonsterSpawner selects:

MonsterDefinition

Compatible Spawn Room

Compatibility is determined using room tags.

Example:

```
MonsterA
  allowed tags:
    dark
    ritual
```

```
MonsterB
  allowed tags:
    nursery
```

Monster Movement

Monster navigates the room graph.

Travel sequence:

```
Current Room Center

→ Exit Point

→ Travel Delay

→ Entry Point

→ Target Room Center
```

Rooms provide navigation markers.

```
MonsterCenter

DoorwayNorth
DoorwaySouth
DoorwayEast
DoorwayWest
```

Monster movement is graph-based.

No NavMesh is required.

Item System

ItemDefinition (ScriptableObject)

Stores static item data.

Examples:

```
Item Id  
  
Display Name  
  
Category  
  
Pickup Score  
  
Unique  
Consumable
```

Examples:

```
Fuse  
Torch  
Safe Key  
Clue  
Pistol  
Battery
```

ItemDefinition contains NO runtime state.

InventoryItem

Runtime inventory entry.

Stores:

```
ItemDefinition  
  
TargetId
```

Example:

```
Safe Key  
  TargetId = Safe_12
```

Inventory

Stores:

```
InventoryItems
```

Supports:

```
AddItem()  
  
RemoveItem()  
  
HasItem()  
  
HasKeyFor()
```

Safe Keys

Safe keys are item instances.

The ItemDefinition is shared:

```
Safe Key
```

Specific keys store:

```
TargetId = Safe_07
```

Safe checks:

```
Inventory.HasKeyFor(Safe_07)
```

Avoid encoding Safe IDs into item names.

Item Pickup

Pickup prefab contains:

```
ItemDefinition  
  
Pickup Sound  
  
Optional TargetId
```

Pickup flow:

```
Play pickup sound  
  
Add inventory item  
  
Award score  
  
Destroy or disable pickup
```

Item Spawning

ItemSpawner owns item placement.

Rooms only provide spawn locations.

Rooms do NOT decide what spawns.

Item Spawn Points

Rooms contain:

```
ItemSpawnPoint
```

Spawn point data:

Occupied

Allowed Categories

Examples:

KeyItem

TaskItem

Weapon

Tool

Consumable

Unique Items

Examples:

Torch

Pistol

Rules:

Spawn once per run

ItemSpawner tracks unique item placement.

Respawnable Items

Examples:

Fuse

Battery

Ammo

Rules:

May be spawned by events

Task Architecture

Tasks are event-driven.

Examples:

Fuse Box

Generator

Safe

Clue

Task objects notify TaskManager.

TaskManager tracks completion.

Room Events

Room-authored interactions are NOT normal item spawns.

Examples:

Broken Bottles

Books On Floor

Collapsed Furniture

Repairable Decorations

These objects:

Exist in room prefab
Start disabled
Become active when event occurs

Use enable/disable.

Do not spawn dynamically.

Example Wine Cellar Event

Initial state:

Shelves Intact
Bottle Pickups Disabled

Monster event:

Shelves Broken
Bottle Pickups Enabled

Player repairs:

Interact Bottles
Hide Bottle Pickup
Increase Progress

Completion:

Shelves Fixed
Award Score

Audio Architecture

Player Audio

Footsteps

Normal Breathing

Heavy Breathing

Voice

Use separate AudioSources.

Monster Audio

Footsteps

Breathing

OneShot

3D audio.

World Audio

Examples:

Doors

Generators

Environmental Sounds

World objects own their AudioSources.

Wall Ownership

Room prefabs own wall visuals.

Generator controls visibility.

Rules:

```
Connected Rooms
  Door opening

Adjacent Rooms
  One wall visible
  One wall hidden

Outside Boundary
  Solid wall or window
```

Avoid overlapping duplicate walls.

Avoid transparent outside wall hacks.

Future Systems

Potential additions:

```
Difficulty Scaling

Monster Variants

Inventory UI

ScriptableObject Room Definitions

ScriptableObject Task Definitions

Advanced Room Events

Barricades

Multiple Monsters
```

Current priority:

Monster Behaviour

Item Spawning

Task Variety

Room Content